Chap 1

Basic Concept

海大資訊工程系
蔡東佐

Outline

- Overview：System Life Cycle ×不太会考
- Pointer and Dynamic Memory Allocation
- Algorithm Specification
- Data Abstraction
- Performance Analysis
- Performance Measurement

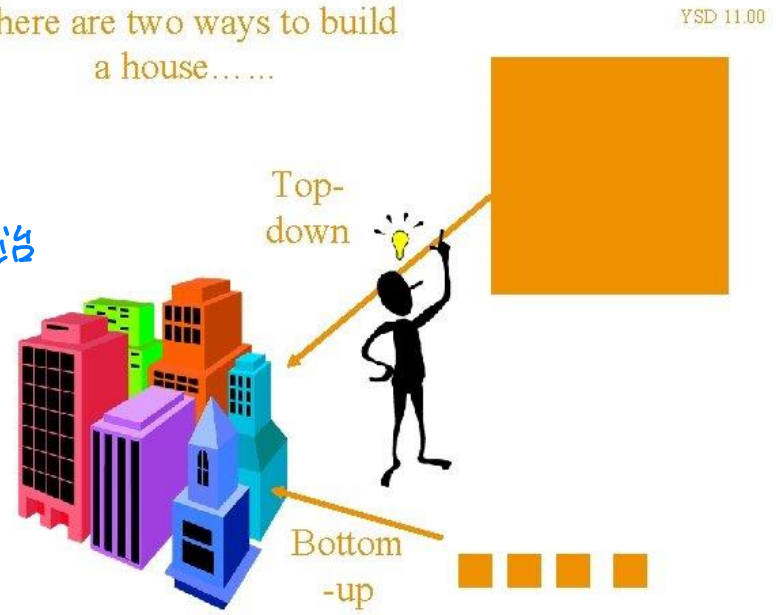
System Life Cycle – Req. and Analysis

軟工4大步驟：1、規格·需求
2、分析·設計
3、実作
4、測試

- Requirements.
  - ✓All large programming projects begin with a set of **spec**ifications that define the purpose of the project.

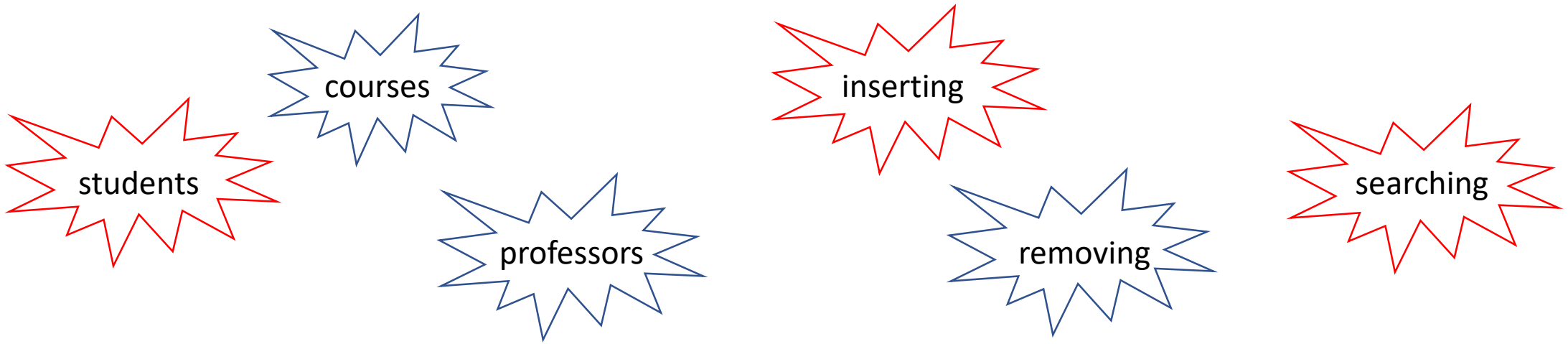
(規格·需求)

- Analysis.
  - ✓In this phase, we begin to break the problem down into manageable pieces. 將"大問題"分治
    - bottom-up:
    - top-down:



System Life Cycle – Design

- Design.
  - ✓The designer approaches the system from the perspectives of both the data objects that the program needs and the operations performed on them. ⇒將資料物件抽象化並設計操作來適配他
    - For example, suppose that we are designing a scheduling system for a university.



System Life Cycle – Refinement and Coding

- Refinement and coding.
  - ✓In this phase, we choose representations for our data objects and write algorithms for each operation on them.

System Life Cycle – Verification

- Verification.
  - ✓This phase consists of developing correctness proofs (正式的證明) for the program, testing the program with a variety of input data, and removing errors.
    - Correctness proofs:
      - Programs can be proven correct using the same techniques that abound in mathematics.
      - Selecting algorithms that have been proven correct can reduce the number of errors.
    - Testing:
      - Testing requires the working code and sets of test data.
    - Error removal:
      - If done properly, the correctness proofs and system tests will indicate erroneous code.

Pointers and Dynamic Memory Allocation

➢Pointers

- ✓Pointers are fundamental to C and C provides extensive support for them.
- ✓The two most important operators used with the pointer type are:
  - & the address operator
  - * the dereferencing (or indirection) operator
- ✓If we have the declaration:
  - int *i*, **pi*;

    where *i* is an integer variable and *pi* is a pointer to an integer.
- ✓If we say:
  - *pi* = &*i*;

    here, &*i* returns the address of *i* and assigns it as the value of *pi*.

Pointers and Dynamic Memory Allocation

- Dynamic Memory Allocation
  - ✓ When you write your program you may not know how much space you will need
  - ✓ Whenever you need a new area of memory, you may call a function, malloc, and request the amount you need.

```
int i, *pi;
float f, *pf;
pi = (int*) malloc(sizeof(int));
pf = (float *) malloc(sizeof(float));
*pi = 1024;
*pf = 3.14;
printf("an integer = &d, a float = %f\n", *pi, *pf);
free(pi);
free(pf);
```

Algorithm Specification – Introduction

- Algorithms exist for many common problems, and designing efficient algorithms plays a crucial role in developing large-scale computer systems.
- Definition: An algorithm is a finite set of instructions that, if followed, accomplishes a particular task. In addition, all algorithms must satisfy the following criteria:
  - ✓ **Input.** There are zero or more quantities that are externally supplied.
  - ✓ **Output.** At least one quantity is produced.
  - ✓ **Definiteness**. Each instruction is clear and unambiguous.
  - ✓ **Finiteness**. If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
  - ✓ **Effectiveness**. Every instruction must be basic enough to be carried out, in principle, by a person using only pencil and paper.

從頭開找到尾，假設發現某一個數字比其下一個數字來的大，就交換。重覆n次。→

Algorithm Specification – Example 1.1 1/5

- Selection sort: Suppose we must devise a program that sorts a set of $n \geq 1$ integers.
  - ✓A simple solution is given by the following:

From those integers that are currently unsorted, find the smallest and place it next in the sorted list.

選擇排序法

Algorithm Specification – Example 1.1 2/5

➢Selection sort: Suppose we must devise a program that sorts a set of n ≥ 1 integers.

✓We assume that the integers are stored in an array, *list*, such that the *i*-th integer is stored in the *i*-th position, *list* [*i*], 0 ≤ i < n.

- Algorithm:

```
for ( i = 0; i < n ; i++ ) {
    Examine list [i] to list[n –1] and
    suppose that the smallest integer is at list [min];

    Interchange list[i] and list [min];
}
```

list

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
	14	7	2	20	13	31	17	25	36

Algorithm Specification – Example 1.1 3/5

- Selection sort: Suppose we must devise a program that sorts a set of $n \geq 1$ integers.

?

  - ✓Algorithm:

```
for ( i = 0; i < n ; i++ ) {
    Examine list [i] to list[n –1] and suppose that the
    smallest integer is at list [min];

    Interchange list[i] and list [min];
}
```

```
void swap (int *x, int *y)
{
   /* both parameters are pointers to ints */
   int temp = *x;     /* declares temp as an int and assigns
                         to it the contents of what x points to */
   *x = *y;     /* stores what y points to into the
                   location where x points */
   *y = temp;   /* places the contents of temp in
                   location pointed to by y */
}
```

Algorithm Specification – Example 1.1 4/5

- Selection sort: Suppose we must devise a program that sorts a set of $n \geq 1$ integers.

  - ✓Algorithm:

```
for ( i = 0; i < n ; i++ ) {
    Examine list [i] to list[n –1] and suppose that the
    smallest integer is at list [min];

    Interchange list[i] and list [min];
}
```

?

list

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
	14	7	2	20	13	31	17	25	36
0	2	7	14	20	13	31	17	25	36
1	2	7	14	20	13	31	17	25	36
2	2	7	13	20	14	31	17	25	36
3	2	7	13	14	20	31	17	25	36
4	2	7	13	14	17	31	20	25	36
5	2	7	13	14	17	20	31	25	36
6	2	7	13	14	17	20	25	31	36
7	2	7	13	14	17	20	25	31	36
8	2	7	13	14	17	20	25	31	36

Algorithm Specification – Example 1.1 5/5

```
#include <stdio.h>
#include <math.h>
#define MAX_SIZE 101
#define SWAP(x, y, t) ((t) = (x), (x) = (y), (y) = (t))
void sort(int [], int );  /*選擇排序*/
void main(void){
    int i, n;
    int list[MAX_SIZE];
    printf("Enter the number of numbers to generate: ");
    scanf("%d", &n);
    if( n < 1 || n > MAX_SIZE){
        fprintf(stderr, "Improper value of n\n");
        exit(EXIT_FAILURE);
    }
    // 隨機產生數值
    for(i = 0; i < n ; i++) {
        list[i] = rand( ) % 1000;
        printf("%d ",list[i]);
    }
```

```
void ( int list[] , int n ){
    for (int i=0; i < n ; i++){
        int minIdx = i;
        for (int j=i ; j < n ; j++){
            if ( list[minIdx] > list[j]){
                minIdx = j;
            }
        }
        if (minIdx == i) continue;
        else Swap(list[i], list[minIdx]);
    }
}
```

Algorithm Specification – Example 1.1 5/5

```
#include <stdio.h>
#include <math.h>
#define MAX_SIZE 101
#define SWAP(x, y, t) ((t) = (x), (x) = (y), (y) = (t))
void sort(int [], int );  /*選擇排序*/
void main(void){
    int i, n;
    int list[MAX_SIZE];
    printf("Enter the number of numbers to generate: ");
    scanf("%d", &n);
    if( n < 1 || n > MAX_SIZE){
        fprintf(stderr, "Improper value of n\n");
        exit(EXIT_FAILURE);
    }
    // 隨機產生數值
    for(i = 0; i < n ; i++) {
        list[i] = rand( ) % 1000;
        printf("%d ",list[i]);
    }
```

```
void sort(int list[], int n) {
    int i, j, min, temp;
    for(i = 0; i < n ; i++)  {
        min = i;
        for(j = i+1 ; j < n ; j++) {
            if(list[j] < list[min]) {
                min = j;
            }
        }
        SWAP(list[i], list[min], temp);
    }
}
```

Algorithm Specification – Example 1.2 1/4

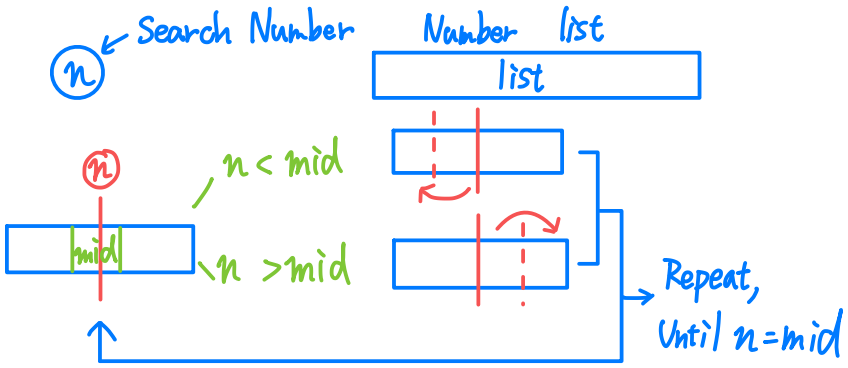
➢Binary search: Assume that we have $n \geq 1$ distinct integers that are already sorted and stored in the array list. That is, list [0] ≤ list [1] ≤ … ≤ list [n-1].

✓ We must figure out if an integer *searchnum* is in this list.

- If it is we should return an index, i, such that list[i] = *searchnum*.
- If *searchnum* is not present, we should return -1.

✓ Algorithm:

```
while (there are more integers to check) {
    middle = (left + right) / 2;
    if (searchnum < list[middle])
            right = middle - 1;
    else if (searchnum == list[middle])
            return middle;
    else left = middle + 1;
}
```



Algorithm Specification – Example 1.2 2/4

- Binary search: Assume that we have $n \geq 1$ distinct integers that are already sorted and stored in the array list. That is, list [0] ≤ list [1] ≤ … ≤ list [n-1].

```
while (there are more integers to check) {
    middle = (left + right) / 2;
    if (searchnum < list[middle])
            right = middle - 1;
    else if (searchnum == list[middle])
            return middle;
    else left = middle + 1;
}
```

Searchnum = 29

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]
	3	11	15	20	23	29	31	35	36	43	47	49	50	53	56
	3	11	15	20	23	29	31	35	36	43	47	49	50	53	56
	3	11	15	20	23	29	31	35	36	43	47	49	50	53	56

Algorithm Specification – Example 1.2 3/4

- Binary search: Assume that we have $n \geq 1$ distinct integers that are already sorted and stored in the array list. That is, list [0] $\leq$ list [1] $\leq \ldots \leq$ list [n-1].

```
while (there are more integers to check) {
    middle = (left + right) / 2;
    if (searchnum < list[middle])
            right = middle - 1;
    else if (searchnum == list[middle])
            return middle;
    else left = middle + 1;
}
```

Searchnum = 43

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]
	3	11	15	20	23	29	31	35	36	43	47	49	50	53
	3	11	15	20	23	29	31	35	36	43	47	49	50	53
	3	11	15	20	23	29	31	35	36	43	47	49	50	53
	3	11	15	20	23	29	31	35	36	43	47	49	50	53

Algorithm Specification – Example 1.2 4/4

```
while (there are more integers to check) {
    middle = (left + right) / 2;
    if (searchnum < list[middle])
            right = middle - 1;
    else if (searchnum == list[middle])
            return middle;
    else left = middle + 1;
}
```

```
#define COMPARE(x,y) (((x) < (y)) ? -1: ((x) == (y))? 0:1)

int binsearch(int list[], int searchnum, int left, int right)
{
    /* searching list[0] <= list[1] <= … <= list[n-1] for
      searchnum . Return its position if found. Otherwise
      return -1 */
    int middle;
    while (list[middle]==searchnum ){
    if    middle = (left+right)/2;
          if (searchnum < list[middle]) right = middle -1;
          else if (searchnum == list[middle] ) break;
          else left = middle+1;
    }
    return -1;
}
```

Algorithm Specification – Example 1.2 4/4

```
while (there are more integers to check) {
    middle = (left + right) / 2;
    if (searchnum < list[middle])
            right = middle - 1;
    else if (searchnum == list[middle])
            return middle;
    else left = middle + 1;
}
```

```
#define COMPARE(x,y) (((x) < (y)) ? -1: ((x) == (y))? 0:1)

int binsearch(int list[], int searchnum, int left, int right)
{
    /* searching list[0] <= list[1] <= … <= list[n-1] for
     searchnum . Return its position if found. Otherwise
     return -1 */
    int middle;
    while (left <= right) {
        middle = (left + right)/2;
        switch (COMPARE(list[middle], searchnum)) {
            case -1: left = middle + 1;
            case 0 : return middle;
            case 1 : right = middle – 1;
        }
    }
    return -1;
}
```

Algorithm Specification – Recursive Algorithms

- Typically, beginning programmers view a function as something that is invoked (called) by another function.
- This perspective ignores the fact.
  - ✓ Functions can call themselves (direct recursion).
  - ✓ Functions may call other functions that invoke the calling function again (indirect recursion).
- How do we determine when we should express an algorithm recursively?

$$\binom{n}{m} = \left(\frac{n!}{m!(n-m)!}\right) \longrightarrow \binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$

Algorithm Specification – Recursive Algorithms

Program 1.7. Searching an ordered list

```
#define COMPARE(x,y) (((x) < (y)) ? -1: ((x) == (y))? 0:1)

int binsearch(int list[], int searchnum, int left, int right)
{
    /* searching list[0] <= list[1] <= … <= list[n-1] for
     searchnum . Return its position if found. Otherwise
     return  -1 */
    int middle;
    while (left <= right) {
        middle = (left + right)/2;
        switch (COMPARE(list[middle], searchnum)) {
            case -1: left = middle + 1;
            case 0 : return middle;
            case 1 : right = middle – 1;
        }
    }
    return -1;
}
```

Program 1.8. Recursive Implementation of Binary Search

```
int binsearch (int list[], int searchnum, int left, int right)
{   /* search list[0]<=list[1]<=…<=list[n-1] for searchnum. Return its
       position if found. Otherwise return -1 */
    int middle;
    if (left <= right) {
        middle = (left + right)/2;
        if (list[middle] < right) binsearch(list, searchnum, middle+1, right);
        else if (list[middle] == searchnum) return middle;
        else binsearch(list, searchnum, left, middle-1);
    }
    return -1;
}
```

Algorithm Specification – Recursive Algorithms

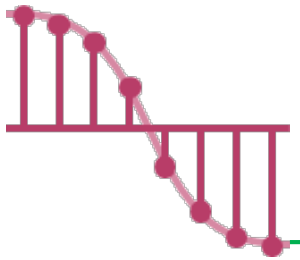
Program 1.7. Searching an ordered list

```
#define COMPARE(x,y) (((x) < (y)) ? -1: ((x) == (y))? 0:1)

int binsearch(int list[], int searchnum, int left, int right)
{
    /* searching list[0] <= list[1] <= … <= list[n-1] for
     searchnum . Return its position if found. Otherwise
     return  -1 */
    int middle;
    while (left <= right) {
        middle = (left + right)/2;
        switch (COMPARE(list[middle], searchnum)) {
            case -1: left = middle + 1;
            case 0 : return middle;
            case 1 : right = middle – 1;
        }
    }
    return -1;
}
```

Program 1.8. Recursive Implementation of Binary Search

```
int binsearch (int list[], int searchnum, int left, int right)
{   /* search list[0]<=list[1]<=…<=list[n-1] for searchnum. Return its
       position if found. Otherwise return -1 */
    int middle;
    if (left <= right) {
        middle = (left + right)/2;
        switch (COMPARE(list[middle], searchnum)) {
            case -1: return binsearch(list, searchnum, middle + 1, right);
            case 0 : return middle;
            case 1 : return binsearch(list, searchnum, left, middle – 1);
        }
    }
    return -1;
}
```



Data Abstraction

- The basic data types of C include char, int, float, and double.
- The real world abstractions we wish to deal with must be represented in terms of these data types.
- In addition to these basic types, C helps us by providing two mechanisms for grouping data together.
  - ✓ Array
    - int list[5] defines a five-element array of integers whose legitimate subscripts are in the range 0 ... 4.
  - ✓ Structure
    - struct student {

      char lastName;

      int studentId;

      char grade;

      }

Data Abstraction – Data Type

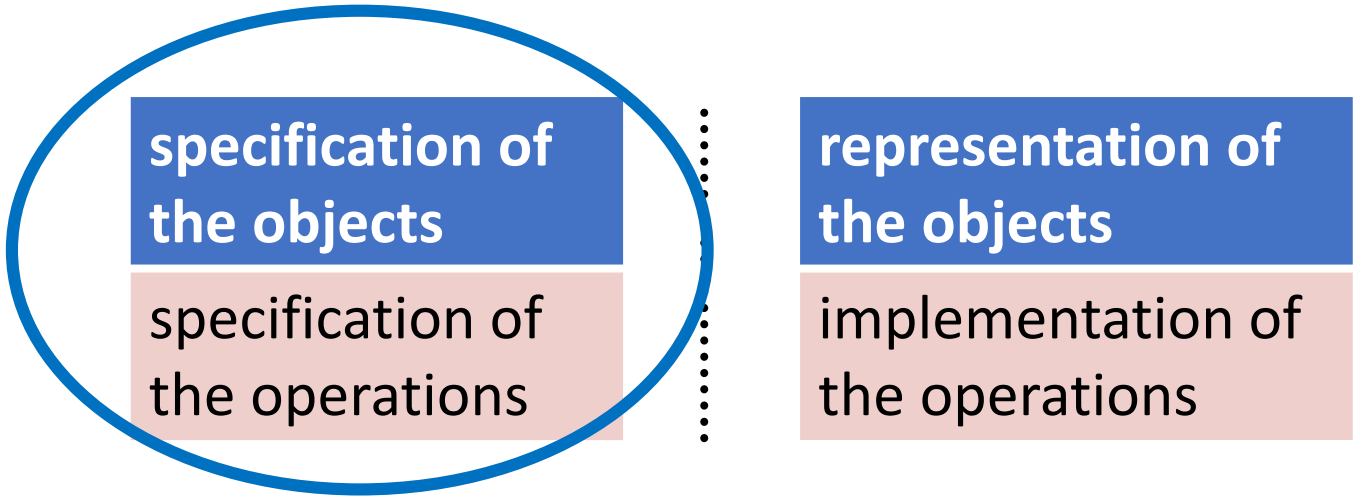
- All programming languages provide at least a minimal set of predefined data types, plus the ability to construct new, or user-defined types.
- **Definition**: A data type is a collection of objects and a set of operations that act on those objects.
  - ✓ Example:
    - The data type **int** consists of the objects {0, +1, -1, +2, -2, …, INT_MAX, INT_MIN} and the operations +, -, *, /, and %.

Data Abstraction – Abstraction Data Type

- It has been observed by many software designers that hiding the representation of objects of a data type from its users is a good design strategy.
- **Definition**: An abstract data type (ADT) is a data type that is organized in such a way that the specification of the objects and the specification of the operations on the objects is separated from the representation of the objects and the implementation of the operations.

不考

抽象：
- 好懂
- 缺乏細節、不具体



→実際実践
- 不易懂
- 具体的程式碼

Data Abstraction – Example

ADT NaturalNumber is

objects: an ordered subrange of the integers **starting at zero**
and ending at the maximum integer (INT_MAX) on the computer

functions:

for all x, y ∈ Nat_Number; TRUE, FALSE ∈ Boolean
and where +, -, <, and == are the usual integer operations.

抽象規格 ⟷ 実際実践

NaturalNumber Zero( )	::=	0
Boolean IsZero(x)	::=	**if** (x) return FALSE **else** return TRUE
Boolean Equal(x, y)	::=	**if** (x = = y) return TRUE **else** return FALSE
NaturalNumber Successor(x)	::=	**if** (x = = INT_MAX) return x **else** return x + 1
NaturalNumber Add(x, y)	::=	**if** (x + y <= INT_MAX) return x + y **else** return INT_MAX
NaturalNumber Subtract(x, y)	::=	**if** (x < y) return 0 **else** return x − y

end NaturalNumber

Performance Analysis

- One of the goals of this book is to develop your skills for making evaluative judgment about programs.

  There are many criteria upon which we can judge a program, including

  - ✓ Does the program meet the original specifications of the task? →程式是否滿足原本的需求？
  - ✓ Does it work correctly? → 程式是否正常工作
  - ✓ Does the program contain documentation that shows how to use it and how it works? →程式是否有良好的說明文件
  - ✓ Does the program effectively use functions to create logical units? → 程式是否有有效率的算法
  - ✓ Is the program's code readable? →程式碼是否可讀性高

- We also can judge a program on more concrete criteria. and so we add two more criteria to our list.

  - ✓ Does the program **efficiently use primary and secondary storage**?
  - ✓ Is the program's **running time acceptable for the task**?

Performance Analysis

- These criteria focus on performance evaluation, which we can loosely divide into two distinct fields.
  - ✓ **Performance analysis**: It focuses on obtaining estimates of time and space that are machine independent.
  - ✓ **Performance measurement**: It focuses on obtaining machine-dependent running times.
- We begin our discussion with definitions of the space and time complexity of a program.
  - ✓ **Definition**:
    - The **space complexity** of a program is the amount of memory that it needs to run to completion.
    - The **time complexity** of a program is the amount of computer time that it needs to run to completion.

Performance Analysis – Space Complexity

不一定会考！ 空間複雑度！

- The space needed by a program is the sum of the following components:
  - ✓**Fixed space requirements**:
    - instruction space (space needed to store the code)
    - space for simple variables, fixed-size structured variables (such as structs), and constants.
  - ✓**Variable space requirements**:
    - This component consists of the space needed by structured variables whose size depends on the particular instance, *I*, of the problem being solved.
    - The variable space requirement of a program *P* working on an instance *I* is denoted $S_p(I)$.

$$S(P) = c + S_P(I)$$

Performance Analysis – Space Complexity (Ex. 1)

$S(P) = c + S_p(I)$

- Program 1.10: Simple arithmetic function
  - ✓ float abc(float a, float b, float c)

```
{
  return a + b + b * c + (a + b - c) / (a + b) + 4.00;
}
```

  - ✓ In this program, fixed Space Requirements (c) = 16.

Type	Name	Number of bytes
parameter: float return address: (used internally)	a, b, c	4 * 3 = 12 4

三个变数：3 记忆体空間

回傳也要一个空間

  - ✓ Variable Space Requirements, $S_{abc}(I) = 0$.

滿变動的变數

Performance Analysis – Space Complexity (Ex. 2)

➢Program 1.11: Iterative function for summing a list of numbers

✓ float sum(float list[ ], int n)

```
{
    float tempsum = 0;
    int i;
    for (i = 0; i<n; i++)
        tempsum += list [i];
    return tempsum;
}
```

✓ In this program, $S_{sum}(I) = 0$ because the fact that when an array is passed as an argument to a function, C interprets it as passing the address of the first element of the array.

Performance Analysis – Space Complexity (Ex. 3)

- Program 1.12: Recursive function for summing a list of numbers
  - ✓ float rsum(float list[ ], int n)

```
{
  if (n) return rsum(list, n-1) + list[n-1];
  return list[0];
}
```

有遞迴!

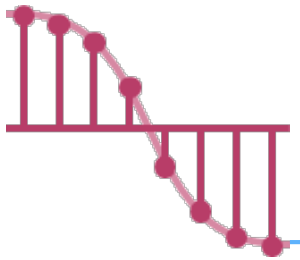
  - ✓ The total variable space needed for the recursive version is $S_{rsum}(I)= 12n$. Thus, the recursive version has a far greater overhead than its iterative counterpart.

Type	Name	Number of bytes
parameter: array pointer parameter: integer return address: (used internally)	list[] n	4 4 4
TOTAL per recursive call		12

Performance Analysis – Time Complexity

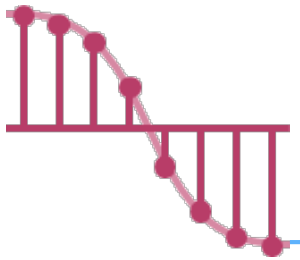
100% 会考！ 時間複雑度

- The time, T(P), taken by a program, P, is the sum of its compile time and its run (or execution) time.
  - ✓ Once we have verified that the program runs correctly, we may run it many times without recompilation.
  - ✓ We are really concerned only with the program's execution time. ← 只考慮執行時間
- Letting n denote the instance characteristic, we might express Tp(n) as
  - ✓ $T_P(n) = c_a ADD(n) + c_s SUB(n) + c_l LDA(n) + c_{st} STA(n)$ → 把所有+-運算的時間全部加起來！
    - c_a, c_s, c_l, c_{st} are constants that refer to the time needed to perform each operation.
    - ADD, SUB, LDA, STA are the number of additions, subtractions, loads and stores.
- Obtaining such a detailed estimate of running time is rarely worth the effort.



Performance Analysis – Time Complexity: Program Step 1/2

- We could count the number of operations the program performs.
  - ✓**Definition**:
    - A program step is a syntactically or semantically meaningful program segment whose execution time is independent of the instance characteristics.
- Example (one program step)
  - ✓a = 2
  - ✓a = 2*b + 3*c /d – e + f /g/a/b/c



Performance Analysis – Time Complexity: Program Step 2/2

- Methods to compute the number of program steps
  - ✓Creating a global variable, *count*.
  - ✓Tabular method
    - Determine the total number of program steps contributed by each statement step per **execution × frequency**
    - Add up the contribution of all statements

Performance Analysis – Time Complexity (Ex. 1) 1/2

➢Count statements Program 1.13 (Program 1.11 with count statements)

```
float sum(float list[ ], int n)
   {
      float tempsum = 0;     count++;              /* for assignment */
      int i;
      for (i = 0; i < n; i++) {
            count++;                               /* for the "for" loop */
            tempsum += list[i];   count++;         /* for assignment */
      }
      count++;                                     /* last execution of "for" */
      count++;                                     /* for return */
      return tempsum;
   }
```

```
tempsum = 0;
i = 0;
tempsum += list[0];
i = 1;
tempsum += list[1];
i = 2;
tempsum += list[2];
i = 3;
return tempsum;
```

if n = 3
count = 2*3+3=9

count = 2n + 3 (steps)

Performance Analysis – Time Complexity (Ex. 1) 2/2

➢Tabular method Figure 1.2 (Step count table for Program 1.11)

steps/execution

Statement	s/e	Frequency	Total Steps
float sum(float list[ ], int n)	0	0	0
{	0	0	0
float tempsum = 0;	**1**	**1**	**1**
int i;	0	0	0
for(i=0; i <n; i++)	**1**	**n+1**	**n+1**
tempsum += list[i];	**1**	**n**	**n**
return tempsum;	**1**	**1**	**1**
}	0	0	0
Total			**2n+3**

Performance Analysis – Time Complexity (Ex. 2) 1/2

➢Program 1.15 (Program 1.12 with count statements added)

```
float rsum(float list[ ], int n)
  {
        count++;            /*for if conditional */
        if (n) {
            count++;        /* for return and rsum invocation */
             return rsum(list, n-1) + list[n-1];
        }
        count++;
        return list[0];
  }
```

```
if(3)
if(2)
if(1)
if(0)
return list[0];
return rsum(list, 0) + list[0];
return rsum(list, 1) + list[1];
return rsum(list, 2) + list[2];
```

if n = 3
count = 2*3+2=8

2n + 2 steps

Performance Analysis – Time Complexity (Ex. 2) 2/2

➢Figure 1.3: Step count table for recursive summing function (Program 1.12)

steps/execution

Statement	s/e	Frequency	Total Steps
float rsum(float list[ ], int n)	0	0	0
{	0	0	0
if (n)	1	n+1	n+1
return rsum(list, n-1)+list[n-1];	1	n	n
return list[0];	1	1 ✓	1
}	0	0	0
Total			2n+2

when 0 L4 Not Execute

Performance Analysis – Asymptotic Notation

- Problem: determining the exact step count of a program could be difficult
  - ✓ **Solution : asymptotic notation (O, Ω, Θ)**
- Complexity of **$c_1n^2+c_2n$** and **c_3n**
  - ✓for sufficiently large of value, c_3n is faster than $c_1n^2+c_2n$
  - ✓**for small values of n, either could be faster**
    - $c_1=1$, $c_2=2$, $c_3=100 \rightarrow$ **$c_1n^2+c_2n \leq c_3n$** for $n \leq 98$
    - $c_1=1$, $c_2=2$, $c_3=1000 \rightarrow$ **$c_1n^2+c_2n \leq c_3n$** for $n \leq 998$
  - ✓No matter what the values of c_1, c_2, and c_3, there will be an n beyond which c_3n is always faster than $c_1n^2+c_2n$

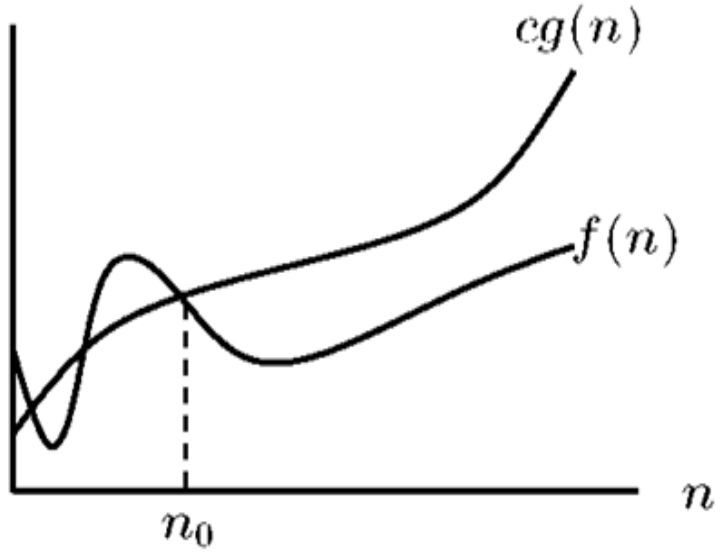
Asymptotic Notation – Big O Notation: O 1/3

➢Definition:

✓ $f(n) = O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \le cg(n)$ for all n, $n \ge n_0$.

- $g(n)$ is a upper bound on $f(n)$. To be informative, $g(n)$ should be as small a function of n as possible for which the statement $f(n) = O(g(n))$ is true.

对於一个式子，只取「对結果影響最大」者作為big O；此外，去掉加、乘常數倍



Examples

$3n+2=O(n)$ 4n, n≥2	/* $3n+2 \le 4n$ for $n \ge 2$ */
$3n+3=O(n)$ 4n, n≥3	/* $3n+3 \le 4n$ for $n \ge 3$ */
$100n+6=O(n)$ 101n, n≥7	/* $100n+6 \le 101n$ for $n \ge 10$ */
$10n^2+4n+2=O(n^2)$ 11n²	/* $10n^2+4n+2 \le 11n^2$ for $n \ge 5$ */
$6*2^n+n^2=O(2^n)$ 7x2^n	/* $6*2^n+n^2 \le 7*2^n$ for $n \ge 4$ */

Asymptotic Notation – Big O Notation: O 2/3

➢ **Theorem 1.2:** If $f(n) = a_m n^m + \ldots + a_1 n + a_0$, then $f(n) = O(n^m)$.

✓ **Proof:**

Definition: f(n) = O(g(n)) iff there exist positive constants c and n_0 such that **$f(n) \le cg(n)$** for all n, $n \ge n_0$.

$$f(n) \le \sum_{i=0}^{m} |a_i| n^i$$

$$= n^m \sum_{i=0}^{m} |a_i| n^{i-m}$$

Since $i - m \le 0, n^{i-m} \le 1$

$$\le n^m \sum_{i=0}^{m} |a_i|, \text{ for } n \ge 1$$

constant

So, $f(n) = O(n^m)$.

Asymptotic Notation – Big O Notation: O 3/3

- The computing times you see the most often in the book.
  - ✓O(1): constant 常數時間
  - ✓O(n): linear 線性時間
  - ✓O(n^2): quadratic 平方時間
  - ✓O(n^3): cubic 立方時間
  - ✓O(2^n): exponential 指數時間
  - ✓O(log$n$): logarithmic
  - ✓O($n$log$n$): log linear

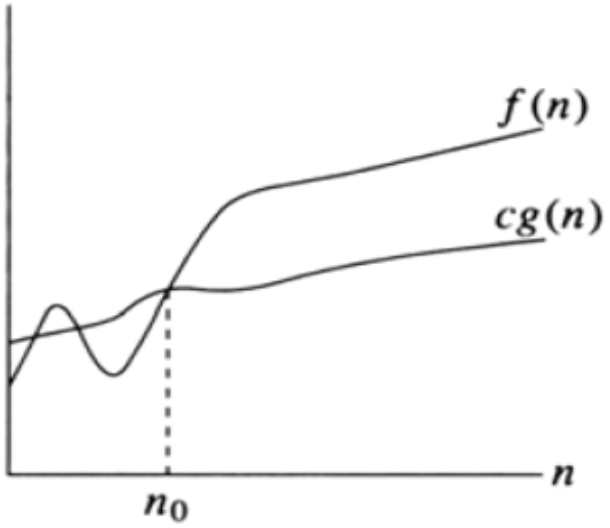
Asymptotic Notation – Omega Notation: Ω

➢ **Definition:**

✓ $f(n) = \Omega(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all n, $n \geq n_0$.

- $g(n)$ is a **lower bound** on $f(n)$. To be informative, $g(n)$ should be as large a function of n as possible for which the statement $f(n) = \Omega(g(n))$ is true.

➢ **Theorem 1.3:** If $f(n) = a_m n^m + \ldots + a_1 n + a_0$ and $a_m > 0$, then $f(n) = \Omega(n^m)$.



Examples

$3n+2 = \Omega(n)$	/* $3n+2 \geq 3n$ for $n \geq 1$ */
$3n+3 = \Omega(n)$	/* $3n+3 \geq 3n$ for $n \geq 1$ */
$100n+6 = \Omega(n)$	/* $100n+6 \geq 100n$ for $n \geq 1$ */
$10n^2+4n+2 = \Omega(n^2)$	/* $10n^2+4n+2 \geq n^2$ for $n \geq 1$ */
$6*2^n+n^2 = \Omega(2^n)$	/* $6*2^n+n^2 \geq 2^n$ for $n \geq 1$ */
$3n+2 = \Omega(1)$	
$10n^2+4n+2 = \Omega(n^2)$	
$10n^2+4n+2 = \Omega(1)$	
$6*2^n+n^2 = \Omega(n)$	

Asymptotic Notation – Theta Notation: Θ 1/2

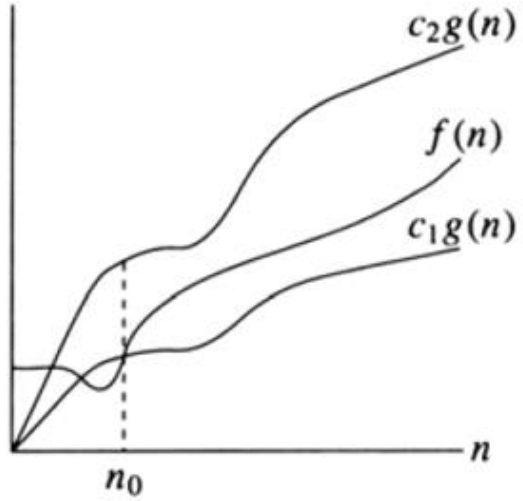
会考

➢ **Definition:**

✓ $f(n) = \Theta(g(n))$ iff there exist positive constants c_1, c_2 and n_0 such that $c_1 g(n) \le f(n) \le c_2 g(n)$ for all n, $n \ge n_0$.

- The theta notation is more precise than both the "big oh" and omega notations.
- **f(n) = Θ(g(n)) iff g(n) is both an upper and lower bound on f(n).**

➢ **Theorem 1.4:** If $f(n) = a_m n^m + \ldots + a_1 n + a_0$ and $a_m > 0$, then $f(n) = \Theta(n^m)$.



Examples

$3n+2 = \Theta(n)$ /* $3n + 2 \ge 3n$ for all $n \ge 2$

$3n + 2 \le 4n$ for all $n \ge 2$

$c_1 = 3$, $c_2 = 4$, and $n_0 = 2$ */

$3n+3 = \Theta(n)$

$10n^2+4n+2 = \Theta(n^2)$

$6*2^n+n^2 = \Theta(2^n)$

Asymptotic Notation – Theta Notation: Θ 2/2

➢Figure 1.5: Step count table for matrix addition

Statement	s/e	Frequency	Total Steps	Asymptotic complexity

Void add (int a[ ]
[MAX_SIZE] …)
{
int i, j;
for (i = 0; i < row; i++)
for (j=0; j< cols; j++)
c[i][j] = a[i][j] + b[i][j];
}
	0
	0
	0
	1
	1
	1
	0
		0
		0
		0
		rows + 1
		rows*(cols+1)
		rows*cols
		0
			0
			0
			0
			rows+1
			rows*cols+rows
			rows*cols
			0
				0
				0
				0
				Θ(rows)
				Θ(rows*cols)
				Θ(rows*cols)
				0

Total			**2rows*cols +2rows+1**	**Θ(rows*cols)**

Function Values

Instance characteristic n							
Time	Name	1	2	4	8	16	32
1	Constant	1	1	1	1	1	1
$\log n$	Logarithmic	0	1	2	3	4	5
n	Linear	1	2	4	8	16	32
$n \log n$	Log linear	0	2	8	24	64	160
n^2	Quadratic	1	4	16	64	256	1024
n^3	Cubic	1	8	64	512	4096	32768
2^n	Exponential	2	4	16	256	65536	4294967296
$n!$	Factorial	1	2	24	40326	20922789888000	26313×10^{33}

Plot of Function Values

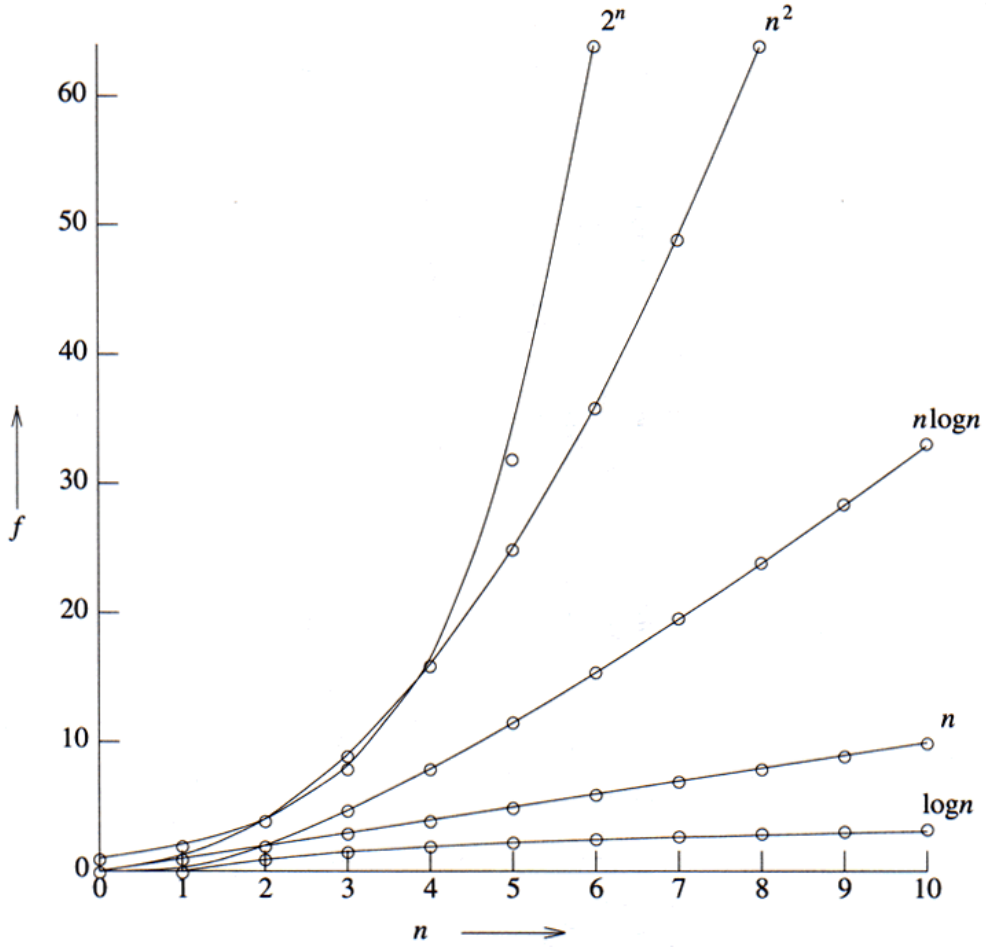
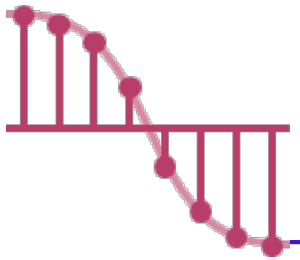


Figure 1.8 Plot of function values

Times on a 1 billion instruction per second computer

	Time for $f(n)$ instructions on a 10^9 instr/sec computer						
n	$f(n)=n$	$f(n)=\log_2 n$	$f(n)=n^2$	$f(n)=n^3$	$f(n)=n^4$	$f(n)=n^{10}$	$f(n)=2^n$
10	.01μs	.03μs	.1μs	1μs	10μs	10sec	1μs
20	.02μs	.09μs	.4μs	8μs	160μs	2.84hr	1ms
30	.03μs	.15μs	.9μs	27μs	810μs	6.83d	1sec
40	.04μs	.21μs	1.6μs	64μs	2.56ms	121.36d	18.3min
50	.05μs	.28μs	2.5μs	125μs	6.25ms	3.1yr	13d
100	.10μs	.66μs	10μs	1ms	100ms	3171yr	$4*10^{13}$yr
1,000	1.00μs	9.96μs	1ms	1sec	16.67min	$3.17*10^{13}$yr	$32*10^{283}$yr
10,000	10.00μs	130.03μs	100ms	16.67min	115.7d	$3.17*10^{23}$yr	
100,000	100.00μs	1.66ms	10sec	11.57d	3171yr	$3.17*10^{33}$yr	
1,000,000	1.00ms	19.92ms	16.67min	31.71yr	$3.17*10^{7}$yr	$3.17*10^{43}$yr	

μs = microsecond = 10^{-6} seconds
ms = millisecond = 10^{-3} seconds
sec = seconds
min = minutes
hr = hours
d = days
yr = years



Performance Measurement

- At some point we also must consider how the algorithm executes on our machine.
- This consideration moves us from the realm of analysis to that of measurement.
- In order to obtain accurate times for short events, we can repeat the programs for several times.

Event timing in C

	Method 1	Method 2
Start timing	start=clock();	start=time(NULL);
Stop timing	stop=clock();	stop=time(NULL);
Type returned	clock_t	time_t
Result in seconds	duration = ((double)(stop-start))/ CLOCKS_PER_SEC;	duration= (double)difftime(stop, start)

Performance Measurement – Example

➢Timing Program for the Selection Sort Function.

```
#include <stido.h>
#include <time.h>
#define MAX_SIZE 1601
#define ITERATIONS 26
#define SWAP(x, y)        { int temp; temp = x; x = y;  y = temp; }
void main(void)
{
    int i, j, position;
    int list[MAX_SIZE];
    int sizelist[] = { 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 200, 300,
                       400, 500, 600, 700, 800, 900, 1000, 1100, 1200, 1300,
                       1400, 1500,1600};
    clock_t start, stop;
    double duration;
     printf(" n time\n");

     for(i=0; i< ITERATIONS; i++) {
            for(j=0; j<sizelist[i]; j++)
                list[j] = sizelist[i]-j; /* worst case */
            start = clock();
            sort(list, sizelist[i]);
            stop = clock();
            /* number of clock ticks per second */
            duration = ((double) (stop-start));
            printf("%6d %f\n", sizelist[i], duration);
     }
}
```